14 haftalık yol haritasında 2. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 02

Algoritmik ve Hesaplamalı Düşünme: Problemden Algoritmaya

Pólya, IPO, sözde kod, akış diyagramı, izleme tablosu ve test

MODÜL I · MAKİNE VE ALGORİTMA · H1–H2

HAFTANIN BÜYÜK SORUSU

Bir bilgisayara çay demlemeyi nasıl öğretirsiniz ve "biraz şeker ekle" komutu neden bir algoritma adımı olamaz?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 2
1.1 Öğrenme çıktıları 3
2. Geçen Haftadan: Makineyi Hatırlayalım 3
3. Kavramsal Anlatım: Problemden Algoritmaya 3
3.1 Problem çözme süreci: Pólya'nın dört adımı 3
3.2 Hesaplamalı düşünme 4
3.3 Girdi–İşlem–Çıktı (IPO) modeli 4
3.4 Algoritma nedir? Knuth'un beş özelliği 5
3.5 Belirsizlik avı: “biraz şeker” neden adım değildir? 5
3.6 Algoritmayı ifade etmenin dört yolu ve sözde kod sözleşmemiz 6
3.7 Akış diyagramı sembolleri (ISO 5807) 6
3.8 Algoritmik düzeyde değişken ve atama 7
3.9 Üç temel kontrol yapısı: sıra, seçim, tekrar 7
3.10 İzleme tablosu, test durumları ve sınır değerleri 9
4. Çözümlü Örnekler: Örnek Merdiveni 9
5. Hata Avcısı 24
6. Mühendis Gözüyle: Doğruluk, Test ve Verimlilik 24
7. Mini-OBS Adımı: v0.2 — Gereksinim Çözümlemesi 24
8. Algoritma Atölyesi ve Alıştırmalar 27
8.1 Atölye etkinlikleri 27
8.2 Alıştırmalar 27
9. Kendini Değerlendirme 29
10. AP-2 Köprüsü ve Merak Kutusu 29
11. Özet, Kavram Haritası ve Terim Sözlüğü 29
12. Kaynaklar 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Birine çay demlemeyi anlatmak kolaydır: “Suyu kaynat, çayı demle, biraz bekle, bardağa koy.” Karşınızdaki insan eksik kalan her şeyi deneyimiyle tamamlar. Aynı cümleleri bir bilgisayara ya da bir robota verdiğinizde ilk sorular hemen gelir: **Ne kadar su? Kaynadığını nasıl anlayacağım? “Biraz” kaç dakika?**

Bu hafta programlamanın en temel becerisini, yani **bir problemi bir makinenin hiçbir yorum yapmadan uygulayabileceği kesin adımlara dönüştürmeyi** öğreneceğiz. Bu dönüşümün adı algoritma tasarımıdır. Geçen hafta işlemcinin komutları birer birer getirip yürüttüğünü gördük; bu hafta o komut dizisinin arkasındaki düşünceyi kuracağız. C dilini 3. haftadan itibaren ayrıntısıyla yazacağız; bu hafta her algoritmanın C karşılığını yalnızca bir **önizleme** olarak göreceğiz.

> GERÇEK DÜNYADAN · **“Algoritma” sözcüğü ve bilinen en eski algoritmalardan biri**
>
> - “Algoritma” sözcüğü, 9. yüzyılda Bağdat'ta çalışmış matematikçi **Muhammed bin Musa el-Hârizmî**'nin adından gelir. Hint–Arap rakamlarıyla hesabı anlatan eserinin Latince çevirilerinde yazarın adı “Algoritmi” biçimini almış; zamanla bu ad, adım adım hesap yöntemlerinin genel adına dönüşmüştür (Knuth, 1997).
> - Bilinen en eski algoritmalardan biri, yaklaşık MÖ 300'de yazılmış **Öklid**'in *Elemanlar* adlı eserindeki, iki sayının en büyük ortak bölenini (EBOB) bulan yöntemdir. Knuth algoritmaları anlattığı başvuru kitabına bu algoritmayla başlar; biz de onu Örnek 12'de yürüteceğiz.

1.1 Öğrenme çıktıları

- **ÖÇ 2.1** Bir problemi Girdi–İşlem–Çıktı (IPO) modeliyle çözümler; kısıtları ve sınır durumlarını belirler.
- **ÖÇ 2.2** Algoritmanın temel özelliklerini (sonluluk, belirlilik, girdi, çıktı, etkinlik) açıklar ve belirsiz adımları tespit eder.
- **ÖÇ 2.3** Problemleri sözde kod ve standart akış diyagramı sembolleriyle ifade eder.
- **ÖÇ 2.4** Sıralı, seçimli ve tekrarlı yapıların (Böhm–Jacopini) algoritmaları ifade etmeye yettiğini örneklerle gösterir.
- **ÖÇ 2.5** Bir algoritmayı izleme tablosu ile elle yürütür ve test durumlarıyla doğrular.

2. Geçen Haftadan: Makineyi Hatırlayalım

Birinci haftada işlemcinin bellekteki komutları **getir–çöz–yürüt** döngüsüyle tek tek işlediğini, C kaynak kodunun bu komutlara **derleme hattı** üzerinden dönüştüğünü ve bellekteki her şeyin **ikili** örüntüler olduğunu gördük. Bu hafta zincirin en başına geçiyoruz: işlemcinin yürüteceği komut dizisini, daha kod yazmadan önce nasıl tasarlarız? Önce aşağıdaki üç soruyu kâğıt üzerinde yanıtlayın:

1. Getir–çöz–yürüt döngüsünde **program sayacı (PC)** hangi bilgiyi tutar?
2. Derleme hattının dört aşamasını sırasıyla yazın: `.c` dosyasından çalıştırılabilir dosyaya.
3. İkili `1011` sayısı onluk tabanda kaçtır? Onluk `13` ikili tabanda nasıl yazılır?

> NOT · **Yanıtlar**
>
> **1.** Bir sonraki komutun bellekteki adresi; atlama komutları PC'ye yeni adres yazar (bugünkü Kaputun Altında'nın anahtarı). **2.** Önişlemci (.i) → derleyici (.s) → çevirici (.o) → bağlayıcı (çalıştırılabilir dosya). **3.** $1011_2 = 11$; $13 = 1101_2$ (sürekli 2'ye bölüp kalanları yazmak da bir algoritmadır).

3. Kavramsal Anlatım: Problemden Algoritmaya

3.1 Problem çözme süreci: Pólya'nın dört adımı

George Pólya, *How to Solve It* (1945) adlı kitabında problem çözmeyi dört adımlı bir süreç olarak tanımlar: **anla, planla, uygula, gözden geçir**. Programlamada problemi anlamak çözümleme (IPO tablosu), planlamak algoritma tasarımı (sözde kod, akış diyagramı), uygulamak kodlama ve elle yürütme, gözden geçirmek ise testtir (sunumdaki ve web sayfasındaki Pólya döngüsü şekli). Kodlamaya erken başlamak çoğunlukla yanlış problemi çözmekle sonuçlanır; bu yüzden her örnekte önce IPO tablosunu, sonra algoritmayı, en son C kodunu göreceksiniz.

Örnek: **"1'den 100'e kadar sayıların toplamı kaçtır?"** *Anla:* girdi N = 100, çıktı tek sayı. *Planla:* tek tek topla (döngü) ya da kısayol ara. *Uygula:* adım adım yürüt. *Gözden geçir:* küçük bir durumla (N = 3 → 6) denetle; "daha kısa yol var mı?" sorusu bizi Örnek 8'deki Gauss formülüne götürür.

3.2 Hesaplamalı düşünme

Jeannette Wing (2006), **hesaplamalı düşünmeyi** (computational thinking) herkese gerekli temel bir beceri olarak tanımlar: problemleri bir bilgisayarın (ya da bir insanın) etkili biçimde yürütebileceği biçimde ifade etmek. Eğitimde bu beceri çoğunlukla dört bileşenle ele alınır: **ayrıştırma** (decomposition) büyük problemi çözülebilir parçalara böler; **örüntü tanıma** (pattern recognition) parçalar arasındaki tekrar eden kalıpları görür; **soyutlama** (abstraction) önemsiz ayrıntıyı atıp özü korur; **algoritma tasarımı** çözümü kesin, sıralı ve sonlu adımlarla yazar.

En sık kullanacağımız araç ayrıştırmadır: Şekil 2.1'de "Sabah okula hazırlan" görevi, her yaprağı tek başına yapılabilecek adımlara bölünmüştür. Wirth'ün (1971) **adım adım iyileştirme** (stepwise refinement) adını verdiği bu yaklaşım H8'de fonksiyonlarla koda dönüşecek.

Şekil 2.1
Ayrıştırma ağacı: "Sabah okula hazırlan" görevinin alt görevlere bölünmesi



turuncu yaprak: Büyük Soru'daki alt problem (o da ayrıca ayrıştırılır)

3.3 Girdi–İşlem–Çıktı (IPO) modeli

Her algoritma dış dünyadan bir şey alır, onu dönüştürür ve bir sonuç verir. **IPO modeli** (Input–Process–Output) bu üç parçayı bir tabloda ayırır (Tablo 2.1). Tablonun altına mutlaka **kısıtlar** (girdilerin geçerli aralıkları, tipleri) ve **sınır durumları** (0, negatif sayı, en büyük değer gibi uç girdiler) eklenir; hataların büyük çoğunluğu bu satırlarda saklanır.

Tablo 2.1
IPO tablosu: iki sayının toplamı ve ortalaması (Örnek 2)

GİRDİ	İŞLEM	ÇIKTI
• a: birinci tamsayı • b: ikinci tamsayı	• toplam ← a + b • ortalama ← toplam / 2	• toplam • ortalama (2 ondalık)

Kısıtlar ve sınır durumları: a ve b tamsayıdır, negatif olabilir; toplam tek ise ortalama kesirlidir (7 + 4 → 5,50). IPO tablosunu kurarken sorun: *Programa ne verilecek, hangi aralıkta? Program ne üretmeli, hangi biçimde? Hangi girdiler geçersizdir, o zaman ne olmalı?* Mini-OBS'nin bu haftaki adımı (Bölüm 7) tam olarak bu tabloyu kurmaktır.

3.4 Algoritma nedir? Knuth'un beş özelliği

Algoritma, bir problemi çözmek için izlenecek, sonlu sayıda, kesin tanımlı adımlar dizisidir. Knuth (1997) bir yöntemin algoritma sayılabilmesi için beş özellik sayar (Tablo 2.2). **Algoritma ile program aynı şey değildir:** algoritma dilden ve makineden bağımsız bir fikirdir; program ise bu fikrin belirli bir dilde (bizim için C) yazılmış, derlenip çalıştırılabilen hâlidir.

Tablo 2.2
Knuth'a göre bir algoritmanın beş özelliği

Özellik	Anlamı	İhlal örneği
Sonluluk (finiteness)	Sonlu sayıda adımdan sonra mutlaka biter.	"i ← 1; i ≠ 10 oldukça i ← i + 2" (i hiç 10 olmaz)
Belirlilik (definiteness)	Her adım kesin ve tek anlamlıdır.	"Biraz şeker ekle", "yeterince pişir"
Girdi (input)	Sıfır ya da daha fazla girdi alır.	Hangi sayılar olduğu belirsiz "en büyük sayıyı bul"
Çıktı (output)	En az bir sonuç üretir.	Hesaplayıp hiçbir şey yazmayan yöntem
Etkinlik (effectiveness)	Her adım kalem–kâğıtla sonlu sürede yapılabilecek kadar temeldir.	"π'nin tüm basamaklarını yaz", "en iyi hamleyi seç"

> MERAK KUTUSU · **Sonluluğu kanıtlamak her zaman kolay değildir**
>
> "n çiftse n ← n / 2, tekse n ← 3n + 1; n = 1 olana kadar yinele." Bu kısa kural denenen her pozitif tamsayı için 1'e ulaşmıştır; ancak **her** pozitif tamsayı için sonlandığı bugün hâlâ kanıtlanamamıştır (Collatz sanısı). Bu yüzden bu haftaki her döngüde "neden biter?" sorusunu açıkça yanıtlayacağız.

3.5 Belirsizlik avı: "biraz şeker" neden adım değildir?

"Biraz şeker ekle" komutu **belirlilik** özelliğini ihlal eder: "biraz" bir kişi için yarım kaşık, bir başkası için üç küptür. Bilgisayarın "biraz"ı yorumlayacak deneyimi yoktur; ona ya kesin bir miktar (2 küp) ya da miktarı belirleyen bir kural ("kullanıcıya kaç küp istediğini sor") vermeliyiz. Tablo 2.3 bir çay tarifini algoritmaya dönüştürürken gereken düzeltmeleri gösterir; "su kaynayınca" gibi ifadelerin aslında gizli birer **tekrar** yapısı olduğuna dikkat edin.

Tablo 2.3
Belirsizlik avı: çay tarifinden algoritmaya

Belirsiz adım	Neden sorunlu?	Kesin (algoritmik) karşılığı
Biraz su koy	Miktar yok	Çaydanlığa 1 L su koy
Su kaynayınca	Nasıl anlaşılacak?	Su kaynadı mı? Hayırsa 1 dakika bekle, yeniden bak (tekrar)
Yeterince çay koy	Ölçü yok	Demliğe 2 tatlı kaşığı (≈ 6 g) çay koy
Bir süre demlensin	Süre yok; sonluluk belirsiz	15 dakika beklet
Biraz şeker ekle	Kişiye göre değişir	Şeker sayısını kullanıcıdan OKU (0–3); o kadar küp ekle
Güzelce karıştır	Ölçülemez	Kaşıkla 10 kez karıştır

3.6 Algoritmayı ifade etmenin dört yolu ve sözde kod sözleşmemiz

Aynı algoritma farklı biçimlerde yazılabilir. **Doğal dil** esnektir ama belirsizliğe açıktır. **Sözde kod** (pseudocode), programlama dillerinin yapısını taşıyan ama sözdizimi kurallarıyla uğraştırmayan kesin bir ara dildir. **Akış diyagramı** (flowchart) aynı yapıyı görsel olarak gösterir. **Programlama dili** ise derleyicinin anlayabildiği tek biçimdir.

Sözde kodun evrensel bir standardı yoktur. Bu derste tüm materyallerde (ders notu, sunum, web, sınavlar) **aynı** sözleşmeyi kullanacağız (Tablo 2.4); son sütun her yapının 3. haftadan itibaren göreceğimiz C karşılığını önizler (hepsinin çalışan hâli: `kodlar/H02_K01_sozlesme_c.c`).

Tablo 2.4
Bu derste kullanılan Türkçe sözde kod sözleşmesi

Sözde kod	Anlamı (akış diyagramındaki sembol)	C karşılığı (önizleme)
BAŞLA … BİTİR	Algoritmanın başı ve sonu (başla/bitir)	`int main(void) { … return 0; }`
OKU x, y	Kullanıcıdan (klavyeden) değer al (giriş/çıkış)	`scanf("%d %d", &x, &y);`
YAZ ifade, "metin"	Ekrana değer ya da metin yaz (giriş/çıkış)	`printf("%d\n", x);`
x ← ifade	Atama: sağ taraf hesaplanır, sonuç x kutusuna yazılır (işlem)	`x = ifade;`
=, ≠, <, ≤, >, ≥	Karşılaştırma (sonuç: doğru/yanlış) (karar içinde)	`==, !=, <, <=, >, >=`
VE, VEYA, DEĞİL	Mantıksal bağlaçlar (karar içinde)	`&&, ||, !`
+, −, ×, /	Aritmetik (/ gerçek bölme) (işlem)	`+, -, *, /`
a DIV b, a MOD b	Tamsayı bölümü ve kalan (a, b ≥ 0) (işlem)	`a / b, a % b (tamsayılarda)`
EĞER k İSE … [DEĞİLSE EĞER k2 İSE …] [DEĞİLSE …] SON	Seçim: koşula göre dallanma (karar)	`if (k) { … } else if (k2) { … } else { … }`
İKEN k YAP … SON	Tekrar: koşul doğru oldukça gövdeyi yinele (önce sına) (karar + geri dönen ok)	`while (k) { … }`
İÇİN i ← 1'DEN n'YE YAP … SON	Sayaçlı tekrar: i ← 1; İKEN i ≤ n … i ← i + 1 kısaltması (karar + geri dönen ok)	`for (int i = 1; i <= n; i = i + 1) { … }`
// açıklama	Okuyucu için not; yürütülmez	`/* açıklama */`

- Her satırda **tek bir adım** yazılır ve adımlar yukarıdan aşağıya yürütülür; bir yapının (EĞER, İKEN, İÇİN) içindeki adımlar **4 boşluk** girintilenir ve yapı **SON** ile kapatılır.
- Anahtar sözcükler BÜYÜK harfle, değişken adları küçük harfli, anlamlı ve Türkçe karaktersiz yazılır (`toplam`, `en_buyuk`); OKU gerekirse bir istem iletisi yazmayı da kapsar.
- **←** atama, **=** karşılaştırmadır (C'de `=` ve `==`; H5'in en sık hatalarından biri). `İÇİN i ← 1'DEN n'YE` yazımında ekler ses uyumuna göre okunur (1'den, n'ye).

3.7 Akış diyagramı sembolleri (ISO 5807)

Akış diyagramı sembolleri **ISO 5807** standardında tanımlanmıştır (ISO, 1985). Kurallarımız: akış **yukarıdan aşağıya** ilerler; her karar elmasının iki dalı **Evet** ve **Hayır** diye etiketlenir; tek bir BAŞLA ve en az bir BİTİR vardır; oklar kesişmez, bir birleşme noktasında (•) buluşur. Kullanacağımız yedi sembol Şekil 2.2'dedir.

Şekil 2.2
ISO 5807 akış diyagramı sembolleri, anlamları ve örnek kullanımları

Sembol	Ad	Anlamı	Örnek
	**Başla / Bitir** *terminal*	Algoritmanın giriş ve çıkış noktası	BAŞLA · BİTİR
	**İşlem** *process*	Hesaplama veya atama	toplam ← a + b
	**Giriş / Çıkış** *data (I/O)*	Veri okuma ya da sonuç yazma	OKU n · YAZ x
	**Karar** *decision*	Koşul sınanır; Evet ya da Hayır dalına gidilir	i ≤ n ?
	**Önceden tanımlı işlem** *predefined process*	Başka yerde tanımlı alt algoritma	Harf notunu belirle
	**Bağlayıcı** *connector*	Kesilen akışı başka noktada sürdürür	A → … → A
	Akış oku *flow line*	Yürütme sırası (yukarıdan aşağıya)	

3.8 Algoritmik düzeyde değişken ve atama

Değişken (variable), üzerinde adı yazılı bir **kutu** gibi düşünülebilir: adı değişmez, içindeki değer değişebilir; kutuda her an **tek** bir değer vardır. **Atama** (assignment) `x ← ifade` biçiminde yazılır ve iki aşamada gerçekleşir: önce sağ taraf o anki değerlerle hesaplanır, sonra sonuç soldaki kutuya yazılır ve eski değer silinir (Şekil 2.3). Bu yüzden `toplam ← toplam + i` bir denklem değil bir **komuttur**. Algoritmalarda iki değişken türü çok sık görülür: her turda bir artan **sayaç** (counter) ve her turda değer biriktiren **toplayıcı** (accumulator). İkisi de döngüden önce doğru **başlangıç değerini** almalıdır.

Şekil 2.3
Etiketli kutu benzetmesi: toplam ← toplam + i atamasının iki aşaması



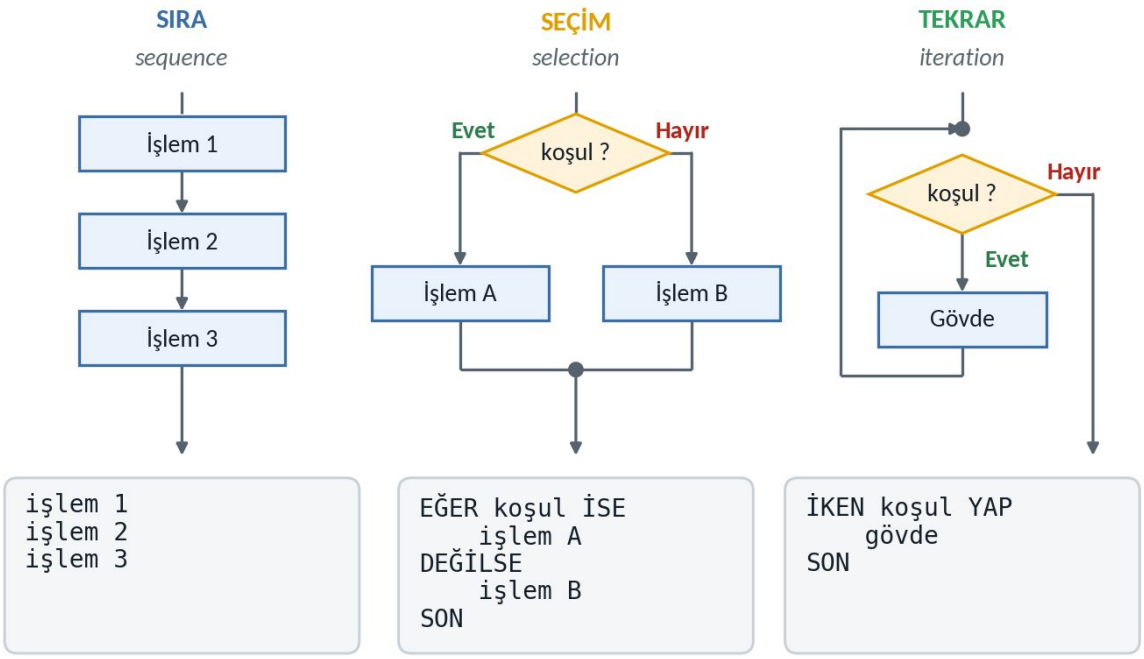
> KAPUTUN ALTINDA · **Kutunun gerçek adresi (H3'e köprü)**
>
> Algoritmik kutu, C'de bellekte belirli sayıda bayt kaplayan bir hücreye dönüşür: her değişkenin bir **adı**, **veri tipi**, **değeri** ve **adresi** vardır. Gelecek hafta kutuların boyutunu `sizeof` ile ölçecek, adreslerini `%p` ile yazdıracağız.

3.9 Üç temel kontrol yapısı: sıra, seçim, tekrar

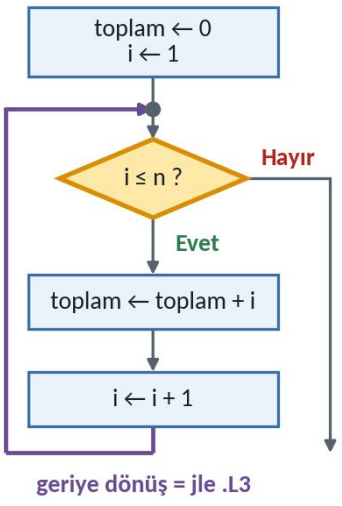
Böhm ve Jacopini (1966), her akış diyagramının yalnızca üç yapıyla ifade edilebileceğini göstermiştir: **sıra** (adımlar birbiri ardına), **seçim** (koşula göre iki yoldan biri) ve **tekrar** (koşul doğru oldukça bir adım grubunu yinelemek). **Yapısal program teoremi** olarak bilinen bu sonuç, Dijkstra'nın (1968) denetimsiz

`goto` kullanımına yönelik eleştirisiyle birlikte yapısal programlamanın temelidir. Her yapının tek girişi ve tek çıkışı olduğundan yapılar iç içe yerleştirilebilir (Şekil 2.4). Örnek merdivenimiz de bu sırayı izler: Örnek 2 yalnızca sıra, 3–6 seçim, 7–12 tekrar ve iç içe yapılar kullanır.

Şekil 2.4
Sıra, seçim ve tekrar yapılarının akış diyagramı ve sözde kod şablonları



Şekil 2.5
Karar = karşılaştır (cmp) + koşullu atla (jle); tekrar = geriye atlama (gerçek gcc çıktısı)




```
gcc -O0 -S -masm=intel · H02_07 döngüsü
  mov DWORD PTR [rbp-16], 0     ; toplam ← 0
  mov DWORD PTR [rbp-12], 1     ; i ← 1
  jmp .L2                       ; önce koşula git
.L3:    ; döngü gövdesi
  mov eax, DWORD PTR [rbp-12]   ; eax ← i
  add DWORD PTR [rbp-16], eax   ; toplam += i
  add DWORD PTR [rbp-12], 1     ; i ← i + 1
.L2:    ; koşul
  mov eax, DWORD PTR [rbp-20]   ; eax ← n
  cmp DWORD PTR [rbp-12], eax   ; i ile n karşılaştır
  jle .L3                       ; i ≤ n ise GERİ atla
```

KAPUTUN ALTINDA · **Karar elması ve döngü işlemcide nedir?**

- İşlemcinin bir "karar" komutu yoktur. ◇ sembolü makine düzeyinde **iki** komuta dönüşür: önce iki değer **karşılaştırılır** (x86-64'te `cmp`) ve durum bayrakları ayarlanır; sonra bir **koşullu atlama** (`jle`: "küçük ya da eşitse atla") bu bayraklara bakarak program sayacına (H1'deki PC) yeni bir adres yazar ya da bir sonraki komutla devam eder.
- **Tekrar** için de özel bir komut yoktur: döngü, programın **geriye doğru** atlamasıdır. Şekil 2.5'teki komutlar Örnek 7'nin `gcc -O0 -S -masm=intel` ile üretilmiş **gerçek** çıktısıdır

> (kodlar/ciktilar/H02_07_toplam_dongu_asm.txt). Döngünün her turunda 6 komut yürütülür (gövdede 3, koşulda 3); N = 1000 için yalnızca döngü ≈ 6 000 komut demektir. Algoritmanın adım sayısı ile yürütülen komut sayısı doğru orantılıdır (x86-64 karşılaştırma ve atlama komutları için bkz. Bryant & O'Hallaron, 2016).

3.10 İzleme tablosu, test durumları ve sınır değerleri

İzleme tablosu (trace table) bir algoritmayı kâğıt üzerinde adım adım yürütmenin disiplinli yoludur: her satır yürütülen bir adımı, her sütun bir değişkeni ya da koşulu gösterir; o adımda değişen hücre vurgulanır (örnek: Tablo 2.6). Henüz değer almamış kutular "–" ile gösterilir; koşul her sınandığında sonucu (Evet/Hayır) yazılır. Yeni başlayanlarda kodu okuma ve izleme becerisi, kod yazma becerisinden önce gelişir (Lister vd., 2004); bu yüzden her yeni yapıyı önce izleyerek öğreneceğiz.

Bir algoritmanın **bir** girdi için doğru çalışması doğru olduğunu göstermez. **Test durumu** (test case), bir girdi ile o girdi için **beklenen** çıktıdan oluşur. İyi bir test kümesi üç tür durum içerir: **normal** (tipik girdiler), **sınır** (aralıkların kenarları; $<$ ile $\leq$ hataları burada çıkar) ve **geçersiz** (kısıt dışı girdiler). Harf notu örneğinde sınır değerleri −1, 0, 59, 60, 69, 70, 79, 80, 89, 90, 100 ve 101'dir. Test tablolarının "Gerçek çıktı" sütunu programın **gerçekten** çalıştırılmasıyla elde edilmiştir.

Aynı probleme farklı algoritmalar. 1'den N'e toplamı N kez toplayarak da tek bir formülle de bulabiliriz. Doğru çözümler arasında **adım sayısı**, **okunabilirlik** ve **genellenebilirlik** ölçütleriyle seçim yaparız; biçimsel verimlilik analizi (O gösterimi) AP-2'nin konusudur, bu hafta yalnızca adımları **sayacağız** (Örnek 5 ve 8).

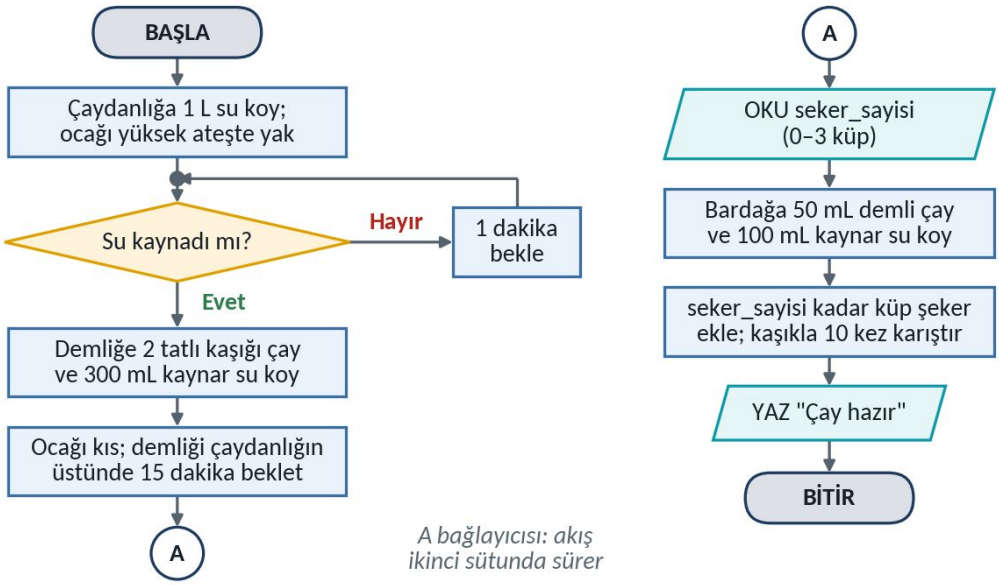
4. Çözümlü Örnekler: Örnek Merdiveni

Örnek merdiveninin her basamağı bir öncekine **tek bir yeni öge** ekler: Örnek 1 belirsizlik avı, 2 sıra, 3–6 seçim, 7–12 tekrar ve iç içe yapılar; **amiral örnekler** (7, 9, 12) standart şablonun dokuz adımıyla işlenir. Tüm C programları `gcc -std=c17 -Wall -Wextra -pedantic` ile **sıfır uyarıyla** derlenmiş, çıktılar gerçek çalıştırmalardan alınmıştır; kod alıntılarındaki satır numaraları kodlar klasöründeki dosyanın gerçek satırlarıdır. C'yi şimdilik sözde kodun satır satır karşılığı olarak okuyun (ayrıntılar H3'ten itibaren; Deitel & Deitel, 2022).

4.1 Örnek 1 ★ — Çay demleme algoritması

Problem. Kesin komutları uygulayabilen ve kullanıcıya soru sorabilen bir robot barista için çay demleme algoritması yazın. **Çözüm.** Tablo 2.3'teki düzeltmeleri akış diyagramına dökelim (Şekil 2.6): "su kaynayınca" bir **tekrar** yapısına, "biraz şeker" ise kullanıcıdan okunan bir **girdiye** dönüşür. Akış iki sütuna sığmadığı için A **bağlayıcısı** kullanılmıştır. **Değiştir:** kullanıcı 5 küp isterse ne olmalı? Kısıtı bir karar olarak ekleyin.

Şekil 2.6
Belirsizliklerinden arındırılmış çay demleme algoritması



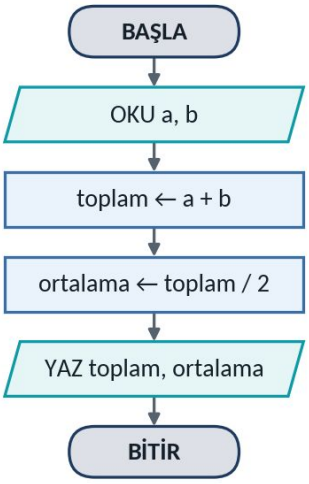
4.2 Örnek 2 ★ — İki sayının toplamı ve ortalaması

Problem. Kullanıcıdan iki tamsayı alıp toplamlarını ve ortalamalarını yazın (IPO: Tablo 2.1). Algoritma yalnızca **sıra** yapısı kullanır: her adım bir kez, yukarıdan aşağıya yürütülür (Şekil 2.7). C önizlemesinde her sözde kod satırı bir C satırına karşılık gelir: OKU → `scanf`, atama → `=`, YAZ → `printf`; `2.0` ile bölmemizin nedeni ortalamanın kesirli olabilmesidir (tamsayı bölmesi H4).

Şekil 2.7
Örnek 2: sıralı yapı

SÖZDE KOD

```
BAŞLA
OKU a, b
toplam ← a + b
ortalama ← toplam / 2
YAZ toplam, ortalama
BİTİR
```



H02_02_toplam_ortalama.c · satır 11–20

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H02_02_toplam_ortalama.c -o toplam_ortalama
```

```c
    int a, b;
    printf("İki tamsayı girin:\n");
    scanf("%d %d", &a, &b);          /* OKU a, b              */

    int toplam = a + b;              /* toplam ← a + b        */
    double ortalama = toplam / 2.0;  /* ortalama ← toplam / 2 */

    printf("Toplam   = %d\n", toplam);       /* YAZ toplam   */
    printf("Ortalama = %.2f\n", ortalama);   /* YAZ ortalama */
    return 0;
```

GERÇEK ÇIKTI (ÖRNEK GİRDİYLE)

```
$ echo "7 4" | ./toplam_ortalama
İki tamsayı girin:
Toplam   = 11
Ortalama = 5.50
```

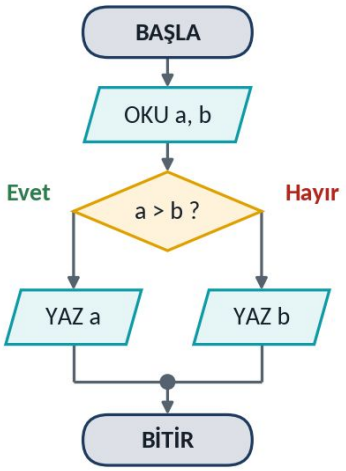
4.3 Örnek 3 ★ — İki sayının büyüğü

Problem. İki tamsayıdan büyük olanı yazın. **Yeni öge:** seçim yapısı. Karar elmasından çıkan iki dal hiçbir zaman aynı anda yürütülmez; hangisinin yürütüleceğini `a > b` koşulu belirler (Şekil 2.8). a = b iken koşul yanlıştır ve DEĞİLSE dalı çalışır; yazılan değer yine doğrudur. Algoritma "hangisi büyük?" sorusuna bir **iletiyle** yanıt verseydi eşitlik ayrıca ele alınmalıydı (Alıştırma 2.4).

Şekil 2.8
Örnek 3: seçim yapısı

SÖZDE KOD

```
BAŞLA
OKU a, b
EĞER a > b İSE
    YAZ a
DEĞİLSE
    YAZ b
SON
BİTİR
```



H02_03_iki_sayi_buyuk.c · satır 15–19

```c
    if (a > b) {                 /* EĞER a > b İSE */
        printf("Büyük olan: %d\n", a);
    } else {                     /* DEĞİLSE */
        printf("Büyük olan: %d\n", b);
    }                            /* SON */
```

GERÇEK ÇIKTI

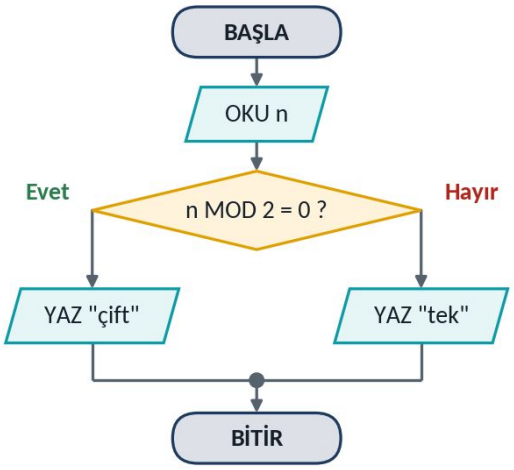
```
$ echo "5 5" | ./iki_sayi_buyuk
İki tamsayı girin:
Büyük olan: 5
```

4.4 Örnek 4 ★ — Sayının tek ya da çift olması

Problem. Bir tamsayının tek mi çift mi olduğunu bulun. **Yeni öge:** kalan (**MOD**) işlemi: 2'ye bölümünden kalan 0 ise sayı çifttir (Şekil 2.9). C'de MOD'un karşılığı `%` işlecidir.

Şekil 2.9
Örnek 4: kalan işlemiyle koşul

```
SÖZDE KOD
BAŞLA
OKU n
EĞER n MOD 2 = 0 İSE
    YAZ "çift"
DEĞİLSE
    YAZ "tek"
SON
BİTİR
```



H02_04_tek_cift.c · satır 14–18

```
14      if (n % 2 == 0) {      /* EĞER n MOD 2 = 0 İSE */
15          printf("%d çift sayıdır\n", n);
16      } else {
17          printf("%d tek sayıdır\n", n);
18      }
```

GERÇEK ÇIKTI

```
$ echo "7" | ./tek_cift
Bir tamsayı girin:
7 tek sayıdır

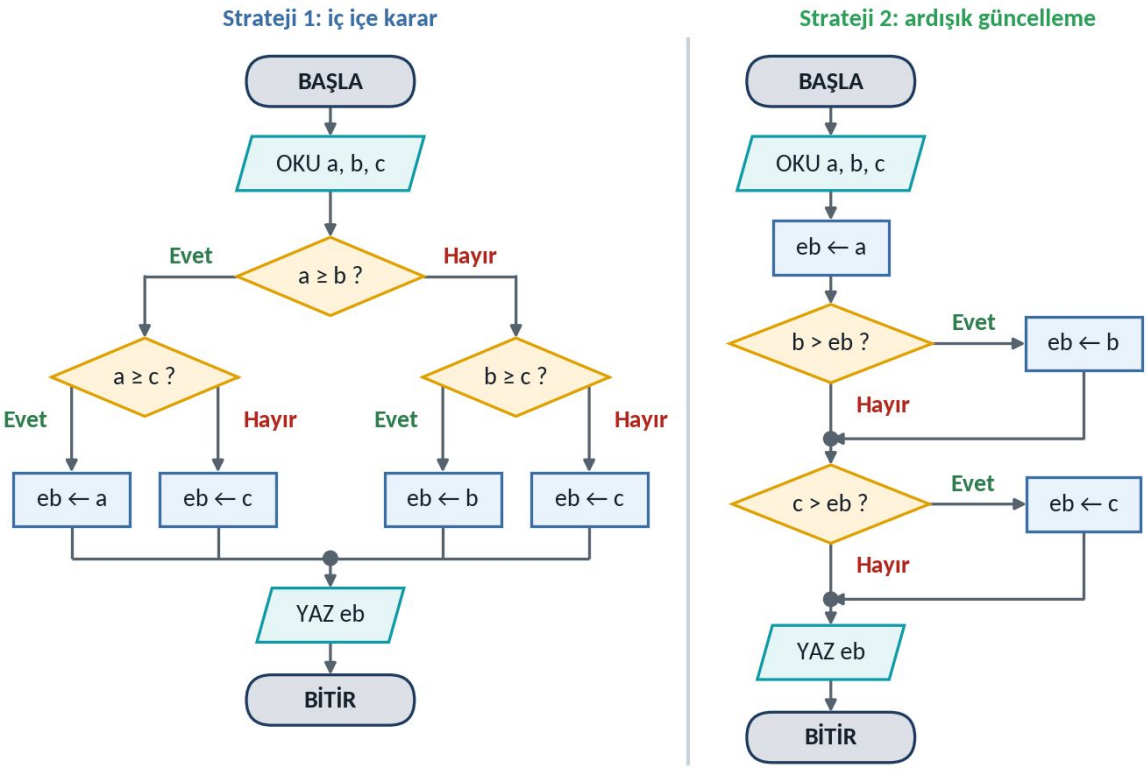
$ echo "-7" | ./tek_cift
Bir tamsayı girin:
-7 tek sayıdır
```

> **HATA AVCISI · Neden "n MOD 2 = 1 ise tek" yazmadık?**
>
> C'de tamsayı bölmesi sıfıra doğru yuvarlar; negatif sayılarda kalan da negatiftir: `-7 % 2` işleminin sonucu −1'dir. "Kalan 1 ise tek" yazsaydık −7 için "çift" yanıtı alırdık. "Kalan 0 ise çift, değilse tek" her tamsayı için doğrudur; negatif sayıyı test kümesine eklemek bu yüzden önemlidir.

4.5 Örnek 5 ★★ — Üç sayının en büyüğü: iki strateji

Problem. Üç tamsayının en büyüğünü bulun. **Yeni öge:** aynı problemi iki stratejiyle çözüp karşılaştırmak (Şekil 2.10). **Strateji 1** (iç içe karar) olasılıkları bir karar ağacıyla ayırır. **Strateji 2** (ardışık güncelleme; C karşılığı aşağıda, 31–38. satırlar) ilk sayıyı "şimdiye kadarki en büyük" kabul eder ve diğerlerini tek tek onunla karşılaştırır: `eb ← a; EĞER b > eb İSE eb ← b SON; EĞER c > eb İSE eb ← c SON`. İkisi de her girdide 2 karşılaştırma yapar; fark **genellenebilirliktedir**: dördüncü bir sayı için strateji 1'in karar ağacı 7 elmasa büyürken strateji 2'ye tek bir EĞER eklenir; N sayı için strateji 2 doğrudan bir döngüye dönüşür (Örnek 9).

Şekil 2.10
Örnek 5: aynı problemin iki stratejisi



H02_05_uc_sayi_en_buyuk.c · satır 31–38

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H02_05_uc_sayi_en_buyuk.c -o uc_sayi_en_buyuk
```

```
31      /* Strateji 2: ardışık güncelleme */
32      int eb2 = a;
33      if (b > eb2) {
34          eb2 = b;
35      }
36      if (c > eb2) {
37          eb2 = c;
38      }
```

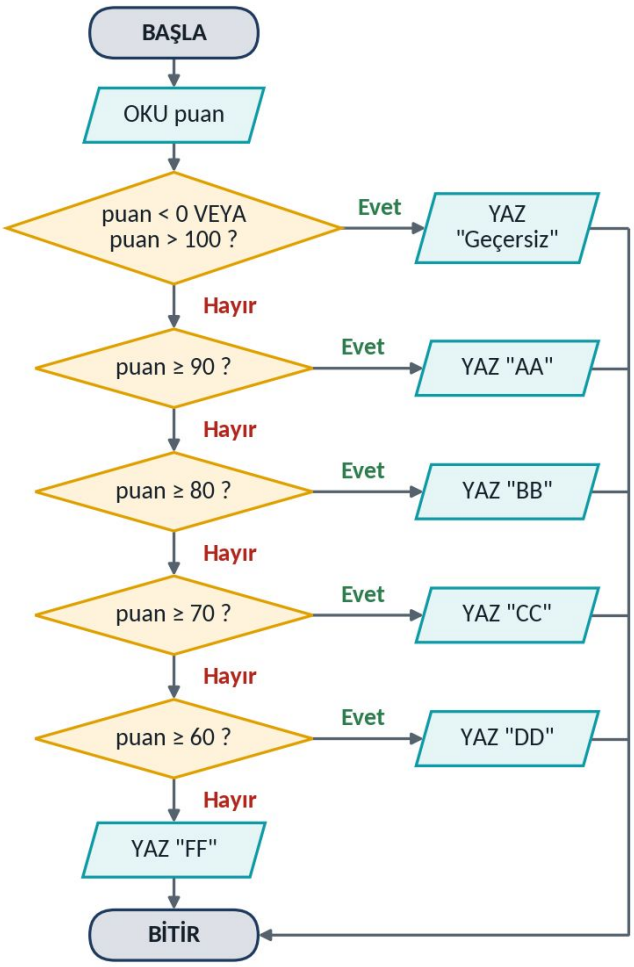
4.6 Örnek 6 ★★ — Puandan harf notu

Problem. 0–100 arasındaki puanı harf notuna çevirin. Basitleştirilmiş bir ölçek kullanıyoruz (90–100 AA, 80–89 BB, 70–79 CC, 60–69 DD, 0–59 FF); yönetmelikteki ölçek farklı olabilir. **Yeni öge:** çok dallı karar ve **aralıkların sıralanması.** Koşullar büyükten küçüğe sınandığı için her karar yalnızca alt sınırı denetler: `puan ≥ 80` koşuluna ulaşıldıysa puanın 90'dan küçük olduğu zaten bellidir (Şekil 2.11). **Değiştir:** koşullar küçükten büyüğe yazılsaydı 95 puan hangi notu alırdı; hangi sınır testi bunu hemen yakalardı?

Şekil 2.11
Örnek 6: çok dallı karar (karar merdiveni)

SÖZDE KOD

```
BAŞLA
OKU puan
EĞER puan < 0 VEYA puan > 100 İSE
    YAZ "Geçersiz puan"
DEĞİLSE EĞER puan ≥ 90 İSE
    YAZ "AA"
DEĞİLSE EĞER puan ≥ 80 İSE
    YAZ "BB"
DEĞİLSE EĞER puan ≥ 70 İSE
    YAZ "CC"
DEĞİLSE EĞER puan ≥ 60 İSE
    YAZ "DD"
DEĞİLSE
    YAZ "FF"
SON
BİTİR
```



H02_06_harf_notu.c · satır 15–21

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H02_06_harf_notu.c -o harf_notu
```

```
15      if (puan < 0 || puan > 100) {
16          printf("Geçersiz puan: %d\n", puan);
17      } else if (puan >= 90) {
18          printf("Harf notu: AA\n");
19      } else if (puan >= 80) {
20          printf("Harf notu: BB\n");
21      } else if (puan >= 70) {
```

Tablo 2.5
Örnek 6 test tablosu (gerçek çıktılar programın çalıştırılmasıyla elde edildi)

Test türü	Girdi	Beklenen	Gerçek çıktı (program)	Sonuç
Normal	85	BB	Harf notu: BB	✓ geçti
Sınır (üst uç)	100	AA	Harf notu: AA	✓ geçti
Sınır 90/89	89	BB	Harf notu: BB	✓ geçti
Sınır 60/59	59	FF	Harf notu: FF	✓ geçti
Geçersiz (alt)	-5	Geçersiz puan	Geçersiz puan: -5	✓ geçti
Geçersiz (üst)	101	Geçersiz puan	Geçersiz puan: 101	✓ geçti

4.7 Örnek 7 ★★ — 1'den N'e kadar toplam (amiral örnek)

1. Problem. Kullanıcının girdiği N için 1 + 2 + … + N toplamını hesaplayın. **Yeni öge:** tekrar yapısı; sayaç ve toplayıcı değişkenleri.

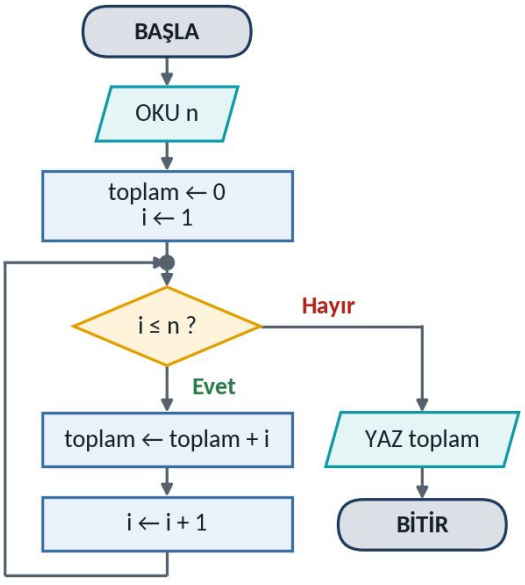
2. Çözümleme. Girdi: n (tamsayı). İşlem: toplam ← 0 ile başla; i = 1, 2, …, n için toplam ← toplam + i. Çıktı: toplam. **Kısıtlar ve sınır durumları:** n = 1 en küçük anlamlı girdidir (toplam 1). n ≤ 0 iken döngü hiç çalışmaz ve 0 yazılır (“boş toplam 0'dır”). Çok büyük n'de toplam C'deki `int` sınırını aşabilir (H3).

3. Algoritma. Toplam tek seferde hesaplanamaz; her turda bir sayı ekleriz. Sıradaki sayıyı tutan **sayaç** `i` ile o ana kadarki toplamı tutan **toplayıcı** `toplam` gerekir; döngü `i ≤ n` doğru oldukça yinelenir (Şekil 2.12). Aynı döngü İÇİN ile de yazılabilir: `İÇİN i ← 1'DEN n'YE YAP toplam ← toplam + i SON` (başlatma, koşul ve artırma tek satırda birleşir).

Şekil 2.12
Örnek 7: tekrar yapısı (İKEN döngüsü)

SÖZDE KOD

```
BAŞLA
OKU n
toplam ← 0     // toplayıcı
i ← 1          // sayaç
İKEN i ≤ n YAP
    toplam ← toplam + i
    i ← i + 1
SON
YAZ toplam
BİTİR
```



4. C kodu (önizleme) ve 7. örnek çalıştırma. C önizlemesi ve n = 3 girdisiyle alınan gerçek çıktı:

H02_07_toplam_dongu.c · satır 15–22

```
15      int toplam = 0;             /* toplam ← 0              */
16      int i = 1;                  /* i ← 1                   */
17      while (i <= n) {            /* İKEN i ≤ n YAP          */
18          toplam = toplam + i;    /*   toplam ← toplam + i   */
19          i = i + 1;              /*   i ← i + 1             */
20      }                           /* SON                     */
21
22      printf("Toplam (1..%d) = %d\n", n, toplam);
```

GERÇEK ÇIKTI

```
$ echo "3" | ./toplam_dongu
N değerini girin:
Toplam (1..3) = 6
```

5. Satır satır açıklama. Yeni olan döngüdür: 15–16. satırlar toplayıcı ve sayacı **başlangıç değeriyle** oluşturur; 17. satırdaki `while (i <= n)` koşulu her turun **başında** sınar (İKEN i ≤ n YAP); 18. satır atamadır; 19. satır sayacı artırır ve koşulun bir gün yanlış olmasını, yani **sonluluğu** sağlar.

6. İzleme tablosu. n = 3 için algoritmayı adım adım yürütelim (Tablo 2.6). Gövde 3 kez, koşul 4 kez sınanır: son sınamada `4 ≤ 3` yanlış çıkar ve döngü biter. n = 5 için aynı tablo 20 satır sürer ve 15 verir.

Tablo 2.6
Örnek 7 izleme tablosu (n = 3)

Adım	İşlem	n	toplam	i	i ≤ n ?	Çıktı
1	OKU n	**3**	–	–		
2	toplam ← 0	3	**0**	–		
3	i ← 1	3	0	**1**		
4	i ≤ n ?	3	0	1	**1 ≤ 3 → Evet**	
5	toplam ← toplam + i	3	**1**	1		
6	i ← i + 1	3	1	**2**		
7	i ≤ n ?	3	1	2	**2 ≤ 3 → Evet**	
8	toplam ← toplam + i	3	**3**	2		
9	i ← i + 1	3	3	**3**		
10	i ≤ n ?	3	3	3	**3 ≤ 3 → Evet**	
11	toplam ← toplam + i	3	**6**	3		
12	i ← i + 1	3	6	**4**		
13	i ≤ n ?	3	6	4	**4 ≤ 3 → Hayır**	
14	YAZ toplam	3	6	4		**6**

8. Test tablosu.

Tablo 2.7
Örnek 7 test tablosu

Test türü	Girdi	Beklenen	Gerçek çıktı (program)	Sonuç
Normal	5	= 15	Toplam (1..5) = 15	✓ geçti
Sınır (en küçük)	1	= 1	Toplam (1..1) = 1	✓ geçti
Sınır (boş toplam)	0	= 0	Toplam (1..0) = 0	✓ geçti
Büyük değer	100	= 5050	Toplam (1..100) = 5050	✓ geçti

9. Değiştir–genişlet. (a) Koşulu `i < n` yaparsanız n = 5 için ne yazılır? (Bölüm 5'teki bir-fark hatası.) (b) Yalnızca tek sayıları toplayın: sayacı 2'şer artırarak ya da gövdeye bir karar ekleyerek. (c) Çarpım biriktirirseniz hangi başlangıç değeri gerekir? → Örnek 10.

4.8 Örnek 8 ★★ — Aynı toplam, Gauss formülüyle

Problem. Örnek 7'deki toplamı döngüsüz hesaplayın ve iki algoritmayı adım sayısıyla karşılaştırın. Anlatılagelen bir öyküye göre genç Carl Friedrich Gauss, 1'den 100'e toplama ödevini sayıları uç uca eşleştirerek hızla çözmüştür; öykünün ayrıntıları tartışmalı olsa da fikir kesindir: 1 + 2 + … + N bir kez düz, bir kez ters sırada alt alta yazılırsa her sütun N + 1 eder (1 + N, 2 + (N − 1), …); N sütun olduğundan iki kat toplam N × (N + 1), dolayısıyla toplam **N × (N + 1) / 2**'dir.

Algoritma yalnızca **sıra** yapısıdır (akış diyagramı sunumda): `BAŞLA · OKU n · toplam ← n × (n + 1) / 2 · YAZ toplam · BİTİR`. C önizlemesi iki algoritmayı aynı programda çalıştırır ve döngünün kaç tur döndüğünü sayar:

H02_08_gauss_toplam.c · satır 17–25

```
17      int toplam = 0, i = 1, tekrar = 0;
18      while (i <= n) {
19          toplam = toplam + i;
20          i = i + 1;
21          tekrar = tekrar + 1;
22      }
23
24      /* Algoritma B: Gauss formülü, tekrar yok */
25      int gauss = n * (n + 1) / 2;
```

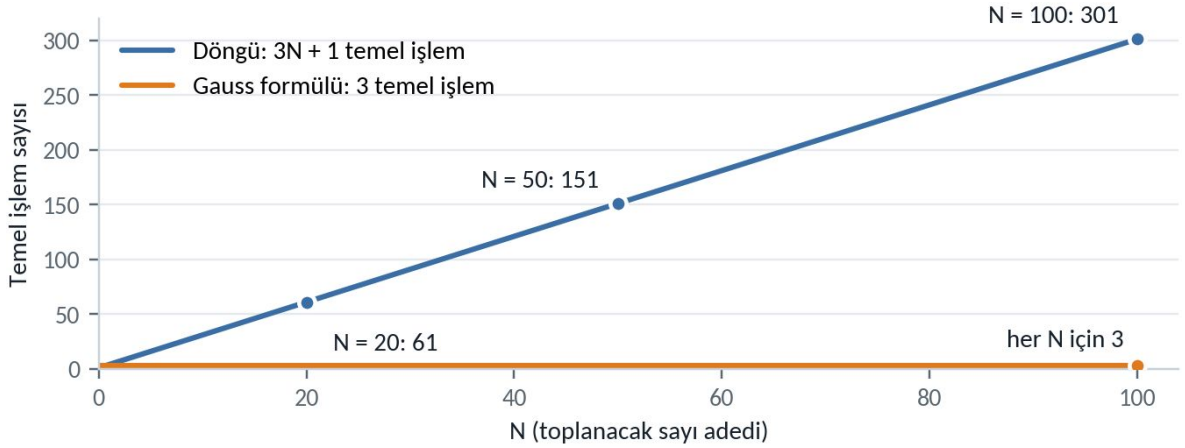
GERÇEK ÇIKTI

```
$ echo "10" | ./gauss_toplam
N değerini girin:
A (döngü): toplam = 55, 10 tur
B (Gauss): toplam = 55, tur yok

$ echo "10000" | ./gauss_toplam
N değerini girin:
A (döngü): toplam = 50005000, 10000 tur
B (Gauss): toplam = 50005000, tur yok
```

İki algoritma da doğru sonucu verir; fark yapılan işin miktarındadır. Karşılaştırma, toplama, çarpma ve bölmeyi **temel işlem** sayarsak döngü her turda bir karşılaştırma, bir toplama ve bir artırma yapar; son karşılaştırmayla birlikte **3N + 1** işlem eder. Formül ise N ne olursa olsun **3** işlem yapar (Şekil 2.13; N = 10 000 için 30 001'e karşı 3). Örnek 8'in assembly çıktısında formül döngüsüz 9 komuttur; döngü ise tur başına 7 komut yürütür (gövde 4: tekrar sayacı da artırılır; koşul 3). N × (N + 1) her zaman çift olduğundan bölme tam çıkar; ama N = 100 000 için ara çarpım 32 bitlik `int` sınırını (2 147 483 647) aşar, programın kısıtı bu yüzden N ≤ 10 000'dir (taşma: H3).

Şekil 2.13
Döngü ve Gauss formülünün N'ye göre temel işlem sayısı



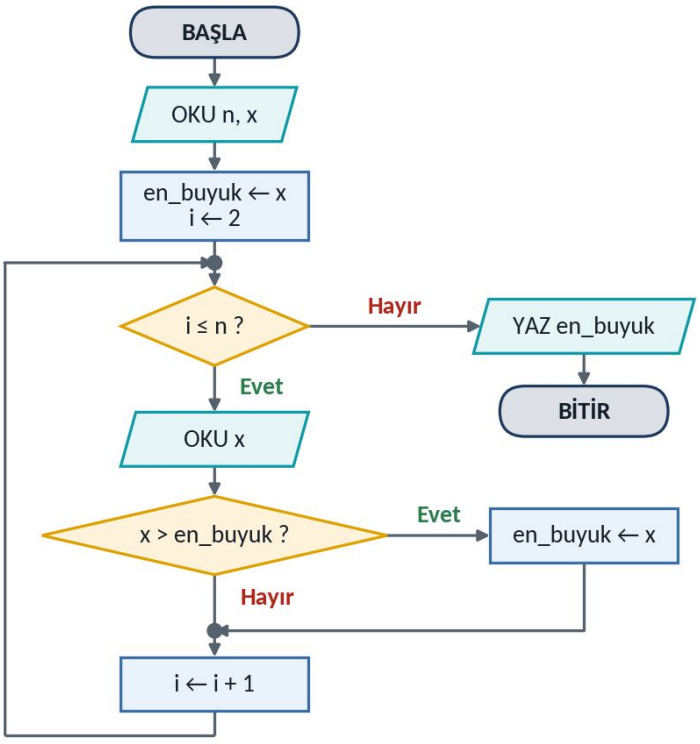
4.9 Örnek 9 ★★ — N sayının en büyüğü (amiral örnek)

1. Problem. Kullanıcı önce kaç sayı gireceğini (N), sonra N tamsayıyı tek tek girer; en büyüğünü bulun. **Yeni öge:** tekrar ve seçimin birlikte kullanılması; **"şimdiye kadarki en iyi"** deseni.

Şekil 2.14
Örnek 9: döngü içinde karar ("şimdiye kadarki en büyük" deseni)

SÖZDE KOD

```
BAŞLA
OKU n, x
en_buyuk ← x  // ilk aday
i ← 2
İKEN i ≤ n YAP
    OKU x
    EĞER x > en_buyuk İSE
        en_buyuk ← x
    SON
    i ← i + 1
SON
YAZ en_buyuk
BİTİR
```



2. Çözümleme. Girdi: n (n ≥ 1) ve x_1 … x_n. İşlem: ilk sayıyı en iyi aday kabul et; her yeni sayıyı adayla karşılaştır, büyükse adayı güncelle. Çıktı: en_buyuk. **Kısıtlar ve sınır durumları:** hiç sayı girilmezse "en büyük" tanımsızdır (n ≥ 1). Sayılar negatif ya da hepsi eşit olabilir; en büyük ilk ya da son sırada gelebilir. Sayıları saklamamız gerekmez: her sayı okunduğu anda karşılaştırılır (diziler H10'da).

3. Algoritma. Örnek 5'in ardışık güncelleme stratejisini bir döngünün içine koyuyoruz (Şekil 2.14). Kritik karar **başlangıç değeridir**: `en_buyuk ← 0` cazip görünür ama tüm sayılar negatifken yanlış sonuç verir; ilk sayıyı aday yapmak her girdi için doğrudur.

4. C kodu (önizleme) ve 7. örnek çalıştırma.

H02_09_n_sayi_en_buyuk.c · satır 17–28

```
17      scanf("%d", &x);
18      int en_buyuk = x;   /* ilk sayı: en iyi aday */
19      int i = 2;
20      while (i <= n) {
21          scanf("%d", &x);
22          if (x > en_buyuk) {   /* daha büyükse güncelle */
23              en_buyuk = x;
24          }
25          i = i + 1;
26      }
27
28      printf("En büyük: %d\n", en_buyuk);
```

GERÇEK ÇIKTI

```
$ echo "5 34 12 58 7 41" |
./n_sayi_en_buyuk
Kaç sayı girilecek?
5 sayı girin:
En büyük: 58
```

5. Satır satır açıklama. 17–18. satırlar ilk sayıyı döngüden önce okuyup aday yapar; bu yüzden sayaç 2'den başlar (19. satır). 22–24. satırlar gövdedeki kararı oluşturur: yeni sayı adaydan büyükse aday güncellenir, değilse hiçbir şey yapılmaz (DEĞİLSE dalı yoktur).

6. İzleme tablosu. Girdi: n = 5 ve 34, 12, 58, 7, 41. Bu desen için her turu tek satırda gösteren **kompakt** tablo daha okunaklıdır (Tablo 2.8). Bir sonraki turda i = 6 olur, `6 ≤ 5` yanlış çıkar ve 58 yazılır; `en_buyuk` yalnızca bir kez değişmiştir.

Tablo 2.8
Örnek 9 kompakt izleme tablosu (her satır bir tur)

i	okunan x	x > en_buyuk ?	en_buyuk
başlangıç	34	–	**34**
2	12	**12 > 34 → Hayır**	34
3	58	**58 > 34 → Evet**	**58**
4	7	**7 > 58 → Hayır**	58
5	41	**41 > 58 → Hayır**	58

8. Test tablosu.

Tablo 2.9
Örnek 9 test tablosu

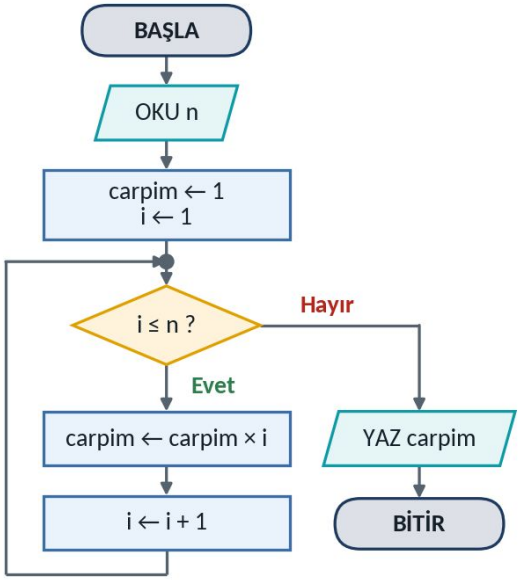
Test türü	Girdi	Beklenen	Gerçek çıktı (program)	Sonuç
Normal	5 34 12 58 7 41	58	En büyük: 58	✓ geçti
Tümü negatif	3 -5 -2 -9	-2	En büyük: -2	✓ geçti
Sınır: tek sayı	1 42	42	En büyük: 42	✓ geçti
Tümü eşit	4 7 7 7 7	7	En büyük: 7	✓ geçti

9. Değiştir–genişlet. (a) En büyüğün kaçıncı sırada girildiğini de yazdırın. (b) En küçüğü aynı döngüde bulun; iki kararı tek bir EĞER–DEĞİLSE EĞER yapısında birleştirmek neden hatalı olabilir? (c) `en_buyuk ← 0` sürümünü "Tümü negatif" testiyle deneyin (Alıştırma 2.5).

4.10 Örnek 10 ★★ — Faktöriyel

Şekil 2.15
Örnek 10: çarpım biriktirme

```
SÖZDE KOD
BAŞLA
OKU n
carpim ← 1    // 0 değil!
İÇİN i ← 1'DEN n'YE YAP
    carpim ← carpim × i
SON
YAZ carpim
BİTİR
```



Problem. n! = 1 × 2 × … × n değerini hesaplayın (0! = 1). **Yeni öge:** çarpım biriktirme ve **başlangıç değerinin önemi**. Toplayıcı 0 ile başlar çünkü 0 toplamanın etkisiz elemanıdır; çarpım ise 1 ile başlamalıdır: `carpim ← 0` yazılırsa sonuç her n için 0 olur. Sözde koddaki İÇİN yapısı akış diyagramında başlatma, koşul ve artırmadan oluşan açık bir döngüdür; C'de `for` satırı bu üç parçayı bir arada yazar (Şekil 2.15).

H02_10_faktoriyel.c · satır 16–20

```
16      int carpim = 1;           /* carpim ← 1 */
17      /* İÇİN i ← 1'DEN n'YE YAP */
18      for (int i = 1; i <= n; i = i + 1) {
19          carpim = carpim * i;  /* carpim ← carpim × i */
20      }                         /* SON */
```

GERÇEK ÇIKTI

```
$ echo "5" | ./faktoriyel
n değerini girin (0-12):
5! = 120

$ echo "0" | ./faktoriyel
n değerini girin (0-12):
0! = 1
```

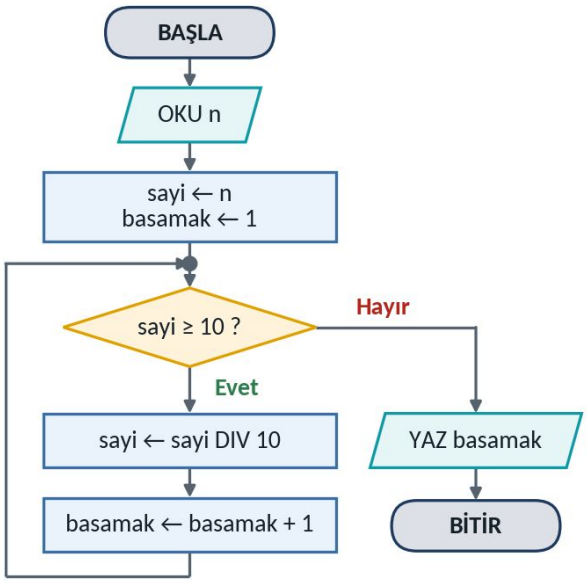
n = 5 için çarpım 1 → 1 → 2 → 6 → 24 → 120 diye büyür. n = 0 için döngü hiç çalışmaz ve çarpım başlangıç değerinde kalır (0! = 1). n = 13 için sonuç (6 227 020 800) `int`'in en büyük değerini aşar; bu yüzden kısıt 0 ≤ n ≤ 12'dir. Taşmanın neden tehlikeli olduğunu gelecek hafta göreceğiz.

4.11 Örnek 11 ★★★ — Bir sayının basamak sayısı

Problem. Negatif olmayan bir tamsayının kaç basamaklı olduğunu bulun. **Yeni öge:** sayaçla değil bir **koşulla** biten döngü ve sonlanmanın gerekçelendirilmesi. Sayıyı 10'a tam bölmek (DIV) son basamağını atar; sayı tek basamaklı kalana kadar kaç kez bölebildiğimizi sayarız (Şekil 2.16, Tablo 2.10).

Şekil 2.16
Örnek 11: tekrarlı bölme

```
SÖZDE KOD
BAŞLA
OKU n          // n ≥ 0
sayi ← n
basamak ← 1
İKEN sayi ≥ 10 YAP
    sayi ← sayi DIV 10
    basamak ← basamak + 1
SON
YAZ basamak
BİTİR
```



Tablo 2.10
Örnek 11 kompakt izleme tablosu (n = 2026)

Tur	sayi ≥ 10 ?	sayi	basamak
başlangıç		**2026**	**1**
1	**2026 ≥ 10 → Evet**	**202**	**2**
2	**202 ≥ 10 → Evet**	**20**	**3**
3	**20 ≥ 10 → Evet**	**2**	**4**
son	**2 ≥ 10 → Hayır**	2	4

Neden sonlanır? Döngü yalnızca `sayi ≥ 10` iken çalışır ve her turda `sayi` en az 10 kat küçülür. Negatif olmayan bir tamsayı sonsuza kadar küçülemeyeceğinden bir noktada `sayi < 10` olur ve döngü biter. `basamak ← 1` seçimi de önemlidir: 0 dahil her sayının en az bir basamağı vardır.

H02_11_basamak_sayisi.c · satır 16–21

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H02_11_basamak_sayisi.c -o basamak_sayisi
```

```
16      int sayi = n;           /* girdinin kopyası */
17      int basamak = 1;        /* 0'ın da 1 basamağı var */
18      while (sayi >= 10) {    /* İKEN sayi ≥ 10 YAP */
19          sayi = sayi / 10;   /* sayi ← sayi DIV 10 */
20          basamak = basamak + 1;
21      }
```

Tablo 2.11
Örnek 11 test tablosu: son satır bir kusuru ortaya çıkarıyor

Test türü	Girdi	Beklenen	Gerçek çıktı (program)	Sonuç
Normal	2026	4 basamaklı	2026 sayısı 4 basamaklıdır	✓ geçti
Sınır: 0	0	1 basamaklı	0 sayısı 1 basamaklıdır	✓ geçti
Sınır: 9 / 10	10	2 basamaklı	10 sayısı 2 basamaklıdır	✓ geçti
Sınır: 100	100	3 basamaklı	100 sayısı 3 basamaklıdır	✓ geçti
Kısıt dışı	-305	3 basamaklı	-305 sayısı 1 basamaklıdır	✗ kaldı

Son test **kaldı**: −305 < 10 olduğundan döngü hiç çalışmaz ve program "1 basamaklı" yazar. Kısıtı (n ≥ 0) açıkça yazmıştık; test tablosu bu kısıtın **neden** gerekli olduğunu gösterdi. Düzeltme: girdiyi doğrulayıp negatif sayıyı reddetmek ya da algoritmanın başına "EĞER n < 0 İSE sayi ← −n" eklemek.

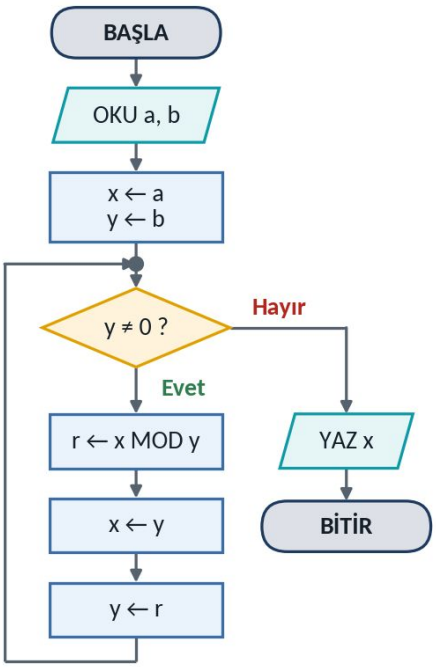
4.12 Örnek 12 ★★★ — Öklid algoritması ile EBOB (amiral örnek)

1. Problem. İki negatif olmayan tamsayının **en büyük ortak bölenini** (EBOB) bulun; örneğin EBOB(48, 18) = 6. **Yeni öge:** 2300 yılı aşkın geçmişi olan klasik bir algoritmayı izlemek ve neden doğru sonlandığını gerekçelendirmek.

Şekil 2.17
Örnek 12: Öklid algoritması

SÖZDE KOD

```
BAŞLA
OKU a, b      // a, b ≥ 0
x ← a
y ← b
İKEN y ≠ 0 YAP
    r ← x MOD y
    x ← y
    y ← r
SON
YAZ x         // EBOB(a, b)
BİTİR
```



2. Çözümleme. Girdi: a, b (a, b ≥ 0; ikisi birden 0 değil). İşlem: b ≠ 0 oldukça r ← a MOD b; a ← b; b ← r. Çıktı: EBOB(a, b). **Kısıtlar ve sınır durumları:** EBOB(a, 0) = a'dır (her sayı 0'ı böler); a < b olabilir; EBOB(0, 0) tanımsızdır; aralarında asal sayılar için sonuç 1'dir.

3. Algoritma. Öklid'in gözlemi: **a'yı ve b'yi bölen her sayı, a'nın b'ye bölümünden kalanı da böler.** Dolayısıyla EBOB(a, b) = EBOB(b, a MOD b) olur ve problem her adımda küçülür (Şekil 2.17). Geometrik yorumu: 48 × 18'lik dikdörtgenden sığabildiği kadar kare kesip kalan parçayla sürdürürsek dikdörtgeni artıksız döşeyen en büyük kare 6 × 6'dır (sunumda ve web sayfasında şekli var).

4. C kodu (önizleme) ve 7. örnek çalıştırma.

H02_12_oklid_ebob.c · satır 16–23

```
16      int x = a, y = b;           /* girdilerin kopyaları */
17      while (y != 0) {            /* İKEN y ≠ 0 YAP  */
18          int r = x % y;          /*   r ← x MOD y   */
19          x = y;                  /*   x ← y         */
20          y = r;                  /*   y ← r         */
21      }                           /* SON             */
22
23      printf("EBOB(%d, %d) = %d\n", a, b, x);
```

GERÇEK ÇIKTI

```
$ echo "48 18" | ./oklid_ebob
Negatif olmayan iki tamsayı
girin:
EBOB(48, 18) = 6
```

5. Satır satır açıklama. Girdiler (a, b) sonuçta yazdırılmak üzere korunur; hesap kopyalar (x, y) üzerinde yapılır (16. satır). Gövdede **sıra önemlidir**: önce kalan r'ye alınmalı, sonra x ← y ve y ← r yapılmalıdır; x ← y önce yazılsaydı eski x kaybolurdu.

6. İzleme tablosu. a = 48, b = 18 için her turu bir satırda gösteren kompakt tablo (Tablo 2.12). Döngü üç tur döner; y = 0 olunca x = 6 yazılır. Her adımın ayrı satırda olduğu ayrıntılı tablo (17 satır) sunumda ve web sayfasında; web sayfasındaki akış diyagramı yürütücüsü aynı yürütmeyi BAŞLA ve BİTİR dahil 19 adımda gösterir.

Tablo 2.12
Örnek 12 kompakt izleme tablosu (a = 48, b = 18)

Tur	x	y	r ← x MOD y	yeni x	yeni y
1	48	18	**48 MOD 18 = 12**	**18**	**12**
2	18	12	**18 MOD 12 = 6**	**12**	**6**
3	12	6	**12 MOD 6 = 0**	**6**	**0**
son	6	0	y = 0 → dur		

8. Test tablosu.

Tablo 2.13
Örnek 12 test tablosu

Test türü	Girdi	Beklenen	Gerçek çıktı (program)	Sonuç
Normal	48 18	= 6	EBOB(48, 18) = 6	✓ geçti
a < b	18 48	= 6	EBOB(18, 48) = 6	✓ geçti
Aralarında asal	17 5	= 1	EBOB(17, 5) = 1	✓ geçti
Sınır: b = 0	7 0	= 7	EBOB(7, 0) = 7	✓ geçti
Sınır: a = 0	0 9	= 9	EBOB(0, 9) = 9	✓ geçti
Büyük sayılar	1071 462	= 21	EBOB(1071, 462) = 21	✓ geçti

Neden sonlanır, neden doğrudur? Her turda yeni y, x MOD y olur ve kalan her zaman bölenden küçüktür (0 ≤ r < y); yani y her turda **kesin azalan** negatif olmayan bir tamsayıdır ve en geç y tur sonra 0'a ulaşır. Her turda EBOB(x, y) korunur; döngü bittiğinde y = 0 olduğundan EBOB(x, 0) = x yazılır. (18, 48) girdisinde ilk tur sayıları yer değiştirir; algoritma a < b durumunu ek bir karar olmadan çözer. **9. Değiştir–genişlet.** Öklid'in özgün çıkarmalı sürümünü yazıp tur sayılarını karşılaştırın (Alıştırma 2.12).

5. Hata Avcısı

Aşağıdaki hataların hiçbiri sözdizimi hatası değildir: algoritma "çalışır" ama yanlış sonuç verir, hiç bitmez ya da uygulanamaz. Bu **mantık hatalarını** derleyici çoğunlukla yakalayamaz; onları izleme tablosu ve iyi seçilmiş test durumları yakalar.

Tablo 2.14
Hata Avcısı: hata, belirti/derleyici iletisi, neden ve çözüm

Hata	Belirti ve derleyici iletisi	Neden	Çözüm
Başlangıç değeri verilmemiş toplayıcı	Rastgele sonuç; `-O0`: **uyarı yok**, `-O2`: "'toplam' is used uninitialized" (HA1)	`toplam ← 0` unutulmuş; kutuda "çöp" değer	Döngüden önce değer ver (toplam 0, çarpım 1)
Hiç yanlış olmayan koşul	Sonsuz döngü; ileti yok	Sayaç değeri atlıyor (i ← i + 2, i ≠ 10)	Eşitsizlik koşulu (i ≤ 10)
Bir-fark hatası (off-by-one)	Uyarı yok; 1..5 toplamı **10** (beklenen 15; HA2)	< ile ≤ karışıklığı	Sınır testleri: n = 0, n = 1
Unutulmuş karar dalı	Eşitlikte yanlış ileti	Yalnızca a > b ve a < b düşünülmüş	Her durum için bir dal ve test
Belirsiz adım	Farklı kişi, farklı sonuç	"biraz", "yeterince", "uygun"	Ölçülebilir kıl ya da girdi al

İlk ve üçüncü satırdaki iletiler `kodlar/hata_avcisi/` klasöründeki iki bilinçli hatalı programın **gerçek** derleme sonuçlarıdır: standart komutla (`-O0`) derleyici başlangıç değeri unutulmuş toplayıcı için **hiçbir uyarı vermez**, yalnızca `-O2` ile uyarır; bir-fark hatasında hiçbir düzeyde uyarı yoktur. **Uyarı olmaması hata olmadığı anlamına gelmez**: bu hataları beklenen değeri bilen testler ve izleme tablosu yakalar (son turda `5 < 5` koşulu yanlış çıkar). `-O2` ile gelen "ignoring return value of 'scanf'" uyarısını H6'da ele alacağız.

6. Mühendis Gözüyle: Doğruluk, Test ve Verimlilik

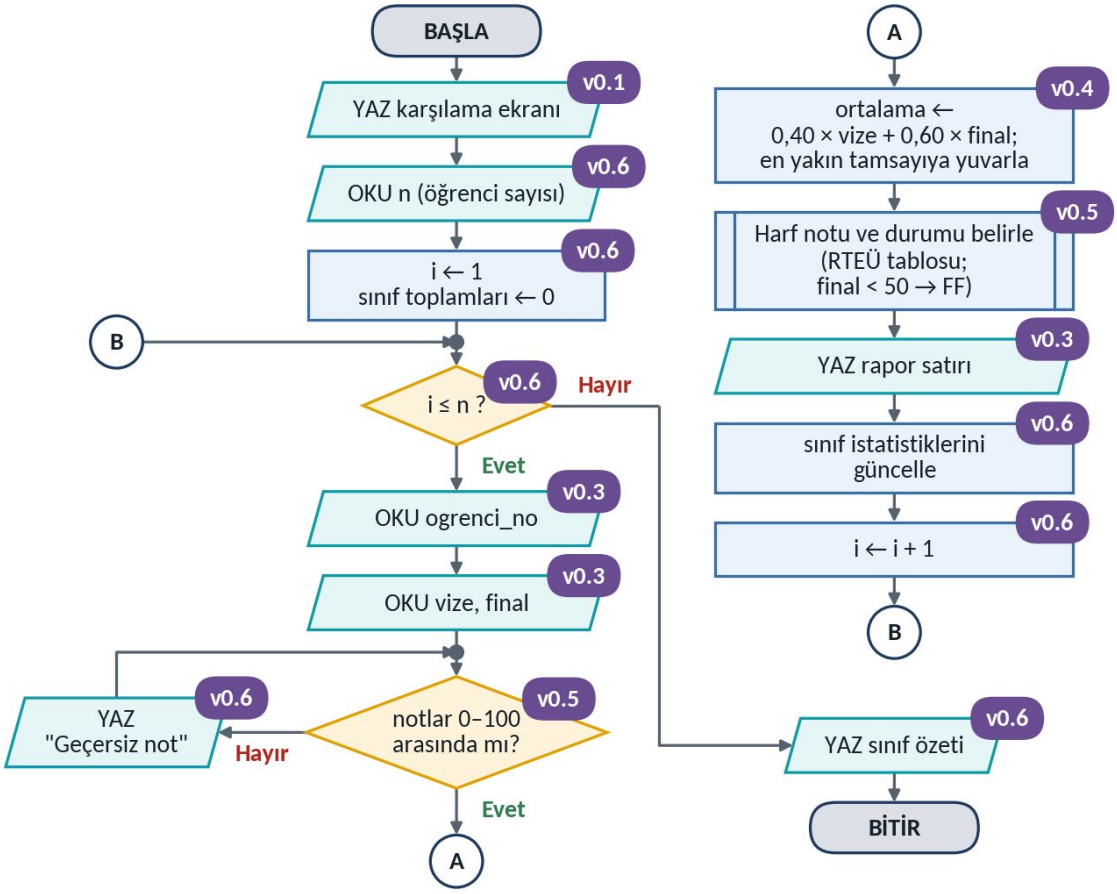
- **Doğruluk ölçülerek gösterilir.** "Benim bilgisayarımda çalışıyor" kanıt değildir; güvenimiz test durumlarının çeşitliliğiyle (normal, sınır, geçersiz) orantılıdır.
- **Sınırlar hataların evidir.** Yanlış bir > / ≥ yalnızca 90 gibi tek bir değerde yanlış sonuç verir; her aralığın iki kenarını ve hemen dışını sınayın.
- **Önce sonlanma, sonra verimlilik.** Her döngü için "hangi değer her turda koşula doğru ilerliyor?" sorusunu yanıtlayın; Örnek 7 ve 8 aynı sonucu verir ama biri N ile büyüyen, diğeri sabit sayıda işlem yapar.
- **Belirsizlik gereksinimlerde başlar.** "Sistem hızlı olmalı" gibi ölçülemeyen gereksinimler "biraz şeker" ile aynı sorunu yaratır.

7. Mini-OBS Adımı: v0.2 — Gereksinim Çözümlemesi

Mini-OBS (Öğrenci Not Yönetim Sistemi) dönem boyunca her hafta o haftanın kavramlarıyla büyüyen projemizdir. H1'de (v0.1) karşılama ekranını tasarladık. **v0.2'de kod yazmıyoruz**; gerçek bir yazılım ekibinin ilk işini yapıyoruz: gereksinimleri çözümleyip sistemin ana akışını çiziyoruz. Not kuralları RTEÜ Ön Lisans ve Lisans Eğitim-Öğretim ve Sınav Yönetmeliği'nin 30. maddesinden alınmıştır (Recep Tayyip Erdoğan Üniversitesi, 2025); v0.4–v0.5'te aynen kodlanacaktır.

- Program önce öğrenci sayısını (n ≥ 1), sonra her öğrenci için **öğrenci numarasını** (10 haneli tamsayı, ör. 2301010045), **vize** ve **final** notlarını alır (0–100, kesirli olabilir: 72,5). Geçersiz notta "Geçersiz not" yazılır ve not **yeniden** sorulur.
- Ortalama **0,40 × vize + 0,60 × final** ile hesaplanır ve en yakın tamsayıya yuvarlanır (59,6 → 60). Harf notu bu değerle RTEÜ'nün dokuz kademeli tablosundan okunur: AA 90–100, BA 81–89, BB 73–80, CB 66–72, CC 60–65, DC 50–59, DD 40–49, FD 30–39, FF 0–29; **final 50'nin altındaysa** harf **FF**'dir (final barajı). CC ve üstü **BAŞARILI**, DC **ŞARTLI BAŞARILI** (YANO ≥ 2,00 ise), DD, FD, FF **BAŞARISIZ**.
- Her öğrenci için bir **rapor satırı** (no, ortalama, yuvarlanmış ortalama, harf, durum), sonunda bir **sınıf özeti** (sınıf ortalaması, en yüksek, en düşük, başarılı sayısı) yazılır.

Şekil 2.18
Mini-OBS v0.2 ana akış diyagramı (mor etiket: bloğun C ile kodlanacağı sürüm; A ve B bağlayıcıları akışın sürdüğü yeri gösterir)



Tablo 2.15
Mini-OBS v0.2 IPO tablosu

GİRDİ	İŞLEM	ÇIKTI
• n: öğrenci sayısı (n ≥ 1) • ogrenci_no: 10 haneli tamsayı (ör. 2301010045) • vize, final: 0–100 (kesirli olabilir)	• notu doğrula; geçersizse "Geçersiz not" yaz, yeniden sor • ortalama ← 0,40 × vize + 0,60 × final; en yakın tamsayıya yuvarla • harf notu ve durum (RTEÜ tablosu; final < 50 → FF) • sınıf istatistiklerini güncelle	• rapor satırı: no · ortalama · yuvarlanmış · harf · durum • sınıf özeti: ortalama, en yüksek, en düşük, başarılı sayısı

Kabul ölçütleri.

- ☐ IPO tablosunda her girdinin tipi ve geçerli aralığı yazılı; en az iki kısıt ve iki sınır durumu belirtilmiş.
- ☐ Ana akış ISO 5807 sembolleriyle çizilmiş; her karar elmasında Evet/Hayır var; tek BAŞLA, en az bir BİTİR; geçersiz not için yeniden okuma döngüsü görünüyor.
- ☐ Ana akışın sözde kodu bu haftanın sözleşmesiyle (Tablo 2.4) yazılmış; her bloğun yanına kodlanacağı sürüm eklenmiş.
- ☐ En az altı test senaryosu (normal, sınır, geçersiz) beklenen çıktılarıyla hazırlanmış; yuvarlama sınırı ve final barajı sınanmış (Tablo 2.16).

Tablo 2.16
Mini-OBS v0.2 test senaryoları (beklenen değerler kurallardan elle hesaplanır)

#	Girdi (no, vize, final)	Beklenen ortalama → yuvarlanmış	Beklenen harf / durum	Tür
1	2301010045, 50, 70	62,00 → 62	CC / BAŞARILI	Normal
2	2301010046, 72,5, 88	81,80 → 82	BA / BAŞARILI	Normal (kesirli not)
3	2301010047, 100, 100	100,00 → 100	AA / BAŞARILI	Sınır (üst uç)
4	2301010048, 59, 60	59,60 → 60	CC / BAŞARILI	Sınır (yuvarlama)
5	2301010049, 69, 53	59,40 → 59	DC / ŞARTLI BAŞARILI (YANO ≥ 2,00 ise)	Sınır (yuvarlama)
6	2301010050, 100, 45	67,00 → 67	FF / BAŞARISIZ (final barajı)	Sınır (final barajı)
7	2301010051, −5, …	—	"Geçersiz not", not yeniden sorulur	Geçersiz

4. ve 5. senaryolar **yuvarlama** sınırını sınar (59,60 → 60 CC; 59,40 → 59 DC); ortalamanın nasıl yuvarlanacağı yönetmelik ve dersin modelleme varsayımıyla verilen bir **gereksinim** kararıdır, C karşılığını H4'te yazacağız. 6. senaryo final barajını sınar: ortalama 67 olsa da final 45 olduğu için harf FF'dir. Örnek 6'daki beş kademeli ölçek yalnızca bir alıştırmaydı (gerçek tablo H5'te). Referans çözüm: `91_Proje_MiniOBS/v0.2/`.

8. Algoritma Atölyesi ve Alıştırmalar

8.1 Atölye etkinlikleri

Etkinlik 1 — Robot oyunu: belirsizlik avı (20 dk). Üçlü gruplar (*Programcı*, *Robot*, *Gözlemci*); malzeme: kapalı tost ekmeği ve kaşar paketleri, plastik bıçak, tabak (gıda kullanılamayan sınıflarda kâğıttan malzemeler). (1) Programcı "peynirli sandviç hazırla" algoritmasını en çok 10 adımda yazar; Robot dışarıda bekler (5 dk). (2) Robot algoritmayı **harfiyen** uygular: ölçülemeyen bir sözcükte "ANLAMADIM: <sözcük>", yapılamayan bir adımda "YAPAMIYORUM" der; Gözlemci duraklamaları not eder (5 dk). (3) Grup algoritmayı yeniden yazar (5 dk). (4) En çok duraklatan adımlar tahtada Knuth'un özellikleriyle eşleştirilir (5 dk). Örnek: "1. Ekmeği al. 2. Üstüne peynir koy. 3. Diğer ekmeği kapat." programında Robot "YAPAMIYORUM: peynir paketi kapalı" (etkinlik) ve "ANLAMADIM: diğer ekmek" (belirlilik) der; düzeltilmiş sürüm paketleri açar, dilim sayısını ve konumları belirtir. Tartışma: "ekmeği örtecek kadar kaşar koy" hangi yapıya dönüşür?

Etkinlik 2 — Eşli çalışma (30 dk). Çiftler şu beş problem için IPO tablosu, sözde kod, akış diyagramı ve izleme tablosu hazırlar, sonra kâğıtları değiştirip birbirlerinin algoritmalarını sınar: (1) Celsius → Fahrenheit (F = C × 9 / 5 + 32); (2) üçgenin eşkenar, ikizkenar ya da çeşitkenar olduğu; (3) 1'den N'e kadar 3'e **veya** 5'e bölünenlerin adedi; (4) FizzBuzz (3'e bölünürse "Fizz", 5'e "Buzz", ikisine "FizzBuzz", değilse sayı); (5) basamakları ters çevirme (1234 → 4321).

Etkinlik 3 — Flowgorithm (20 dk). Ücretsiz Flowgorithm aracında Örnek 7 ve 12'yi çizip değişken izleme penceresiyle yürütün; değerler Tablo 2.6 ve Tablo 2.12 ile aynı sırada değişmelidir.

8.2 Alıştırmalar

Türler: **A)** çıktı tahmini ve izleme, **B)** hata bulma, **C)** Parsons (karışık satırları sıralama), **D)** tamamlama, **E)** algoritma yazma. Önce sözde kod ve akış diyagramıyla çalışın; C önizlemesi isteğe bağlıdır. Çözümler öğrenci sürümünde yer almaz.

Alıştırma 2.1 ★☆☆ *Çıktı tahmini*

Aşağıdaki sözde kodu izleme tablosuyla yürütün ve çıktıyı yazın.

```
x ← 5
y ← 3
x ← x + y
y ← x − y
x ← x − y
YAZ x, y
```

Alıştırma 2.2 ★★☆ *Çıktı tahmini*

Aşağıdaki algoritma için izleme tablosunu (i, s, koşul) kurun. Çıktı nedir? Döngü gövdesi kaç kez çalışır?

```
s ← 0
i ← 1
İKEN i ≤ 10 YAP
    s ← s + i
    i ← i + 3
SON
YAZ s, i
```

Alıştırma 2.3 ★★★ *Çıktı tahmini*

Aşağıdaki "gizemli" algoritma n = 4729 için ne yazdırır? Genel olarak ne hesaplar? Sonlandığını gerekçelendirin.

```
OKU n            // n ≥ 0
t ← 0
İKEN n > 0 YAP
    t ← t + n MOD 10
    n ← n DIV 10
SON
YAZ t
```

Alıştırma 2.4 ★☆☆ *Hata bul*

Aşağıdaki algoritma a = 7, b = 7 için yanlış bir ileti yazar. Hangi karar dalı unutulmuştur? Düzeltin.

```
OKU a, b
EĞER a > b İSE
    YAZ "a daha büyük"
DEĞİLSE
    YAZ "b daha büyük"
SON
```

Alıştırma 2.5 ★★☆ *Hata bul*

"N sayının en büyüğü" algoritmasının aşağıdaki sürümünde başlangıç değeri `en_buyuk ← 0` seçilmiştir. Algoritma hangi girdi için yanlış sonuç verir? Neden? İki farklı düzeltme önerin.

```
OKU n
en_buyuk ← 0
İÇİN i ← 1'DEN n'YE YAP
    OKU x
    EĞER x > en_buyuk İSE
        en_buyuk ← x
    SON
SON
YAZ en_buyuk
```

Alıştırma 2.6 ★★☆ *Hata bul*

Aşağıdaki algoritma 1'den 10'a kadar tek sayıları yazdırmak için yazılmıştır. İzleme tablosuyla birkaç tur yürütün: ne olur? Düzeltin.

```
i ← 1
İKEN i ≠ 10 YAP
    YAZ i
    i ← i + 2
SON
```

Alıştırma 2.7 ★★☆ *Parsons*

Aşağıdaki karışık satırları doğru sıraya ve girintiye koyarak n! (faktöriyel) hesaplayan algoritmayı kurun. Bir satır fazladır (çeldirici); onu bulun.

```
carpim ← carpim × i
YAZ carpim
İKEN i ≤ n YAP
carpim ← 0
OKU n
i ← i + 1
carpim ← 1
SON
i ← 1
```

Alıştırma 2.8 ★☆☆ *Kod tamamla*

Bir tamsayının pozitif, negatif ya da sıfır olduğunu yazan algoritmadaki boşlukları (___) doldurun; akış diyagramını çizin.

```
OKU n
EĞER n ___ 0 İSE
    YAZ "pozitif"
DEĞİLSE EĞER n ___ 0 İSE
    YAZ "negatif"
___
    YAZ "sıfır"
SON
```

Alıştırma 2.9 ★★☆ *Kod tamamla*

N adet sınav puanının ortalamasını bulan algoritmayı tamamlayın; N = 4 ve puanlar 70, 85, 90, 55 için izleme tablosunu kurun.

```
OKU n
toplam ← ___
i ← 1
İKEN i ___ n YAP
    OKU puan
    toplam ← ___
    i ← ___
SON
ortalama ← ___
YAZ ortalama
```

Alıştırma 2.10 ★★☆ *Algoritma yaz*

Bir yılın artık yıl olup olmadığını bulan algoritmayı yazın. Kural: 4'e bölünen yıllar artıktır; ancak 100'e bölünüp 400'e bölünmeyenler artık değildir. Sözde kod, akış diyagramı ve en az 5 satırlık test tablosu hazırlayın.

Alıştırma 2.11 ★★☆ *Algoritma yaz*

1'den N'e kadar olan **çift** sayıların toplamını önce döngüyle, sonra döngüsüz bir formülle hesaplayan iki algoritma yazın. N = 10 ve N = 11 için ikisinin aynı sonucu verdiğini gösterin; N = 1000 için temel işlem sayılarını karşılaştırın.

Alıştırma 2.12 ★★★ *Algoritma yaz*

Öklid'in kitabındaki özgün fikir kalan yerine **çıkarma** kullanır: "a ≠ b oldukça büyükten küçüğü çıkar; sonunda kalan değer EBOB'dur." Bu algoritmayı sözde kodla yazın; (48, 18) ve (100, 1) için döngü tur sayılarını MOD kullanan Örnek 12 ile karşılaştırın. Hangi girdi çiftinde bu sürüm sonlanmaz?

9. Kendini Değerlendirme

- ☐ Bir problemi IPO tablosuyla çözümleyip kısıtları ve sınır durumlarını yazabiliyorum (3.3, 7).
- ☐ Knuth'un beş özelliğini sayıp ihlal örneği verebiliyor; belirsiz adımları ölçülebilir kılabiliyorum (3.4, 3.5).
- ☐ Sözde kod ve ISO 5807 akış diyagramı yazabiliyor; sıra, seçim, tekrarın işlemcideki karşılığını açıklayabiliyorum (3.6–3.9).
- ☐ Bir döngüyü izleme tablosuyla yürütüp normal, sınır ve geçersiz girdilerle test edebiliyorum (3.10, Örnek 7, 12).
- ☐ İki algoritmayı adım sayısıyla karşılaştırıp bir döngünün neden sonlandığını gerekçelendirebiliyorum (Örnek 8, 11, 12).

10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · **Bu haftanın yapı taşları AP-2'de nerede?**

- **Adım sayısı karşılaştırması** (Örnek 5, 8): asimptotik karmaşıklık analizinin sezgisel ön hazırlığı.
- **"N sayının en büyüğü" deseni** (Örnek 9): seçme sıralamasının (selection sort) yapı taşı.
- **Öklid algoritması** (Örnek 12): doğruluk (döngü değişmezi) ve verimlilik tartışmalarının klasik örneği.

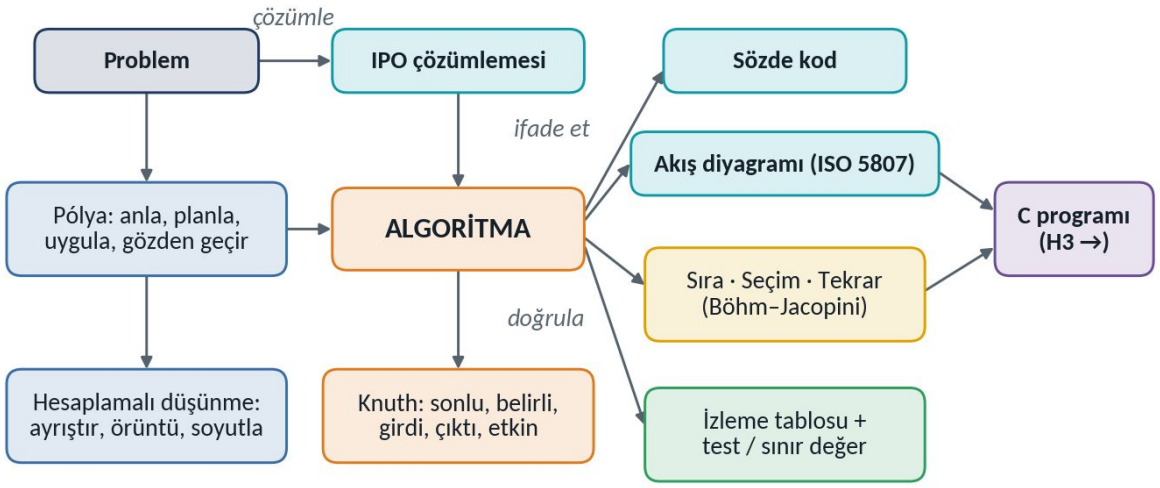
11. Özet, Kavram Haritası ve Terim Sözlüğü

Büyük Soru'ya dönüş. Bilgisayara çay demlemeyi öğretmek için problemi **anlar** ve IPO tablosuyla girdiyi (şeker sayısı), işlemleri ve çıktıyı ayırırız; görevi **ayrıştırır**, her adımı ölçülebilir kılarız; "su kaynayınca" ifadesini **tekrar**, "istersen şeker ekle" ifadesini **seçim** yapısına çeviririz. "Biraz şeker ekle" bir algoritma adımı olamaz, çünkü **belirlilik** özelliğini ihlal eder: bilgisayarın "biraz"ı yorumlayacak deneyimi yoktur; miktar ya kesin bir sayıyla verilmeli ya da kullanıcıdan girdi olarak alınmalıdır.

Tablo 2.17
Hafta 2 terimleri (Türkçe–İngilizce)

Türkçe	İngilizce	Kısa tanım
algoritma	algorithm	Bir problemi çözen sonlu, kesin adımlar dizisi
hesaplamalı düşünme	computational thinking	Problemi bilgisayarın yürütebileceği biçimde ifade etme
ayrıştırma; soyutlama	decomposition; abstraction	Parçalara bölme; önemsiz ayrıntıyı atma
örüntü tanıma	pattern recognition	Parçalar arasındaki benzerliği görme
sözde kod; akış diyagramı	pseudocode; flowchart	Dilden bağımsız kesin yazım; ISO 5807 gösterimi
değişken; atama	variable; assignment	Adı olan değer kutusu; kutuya değer yazma (←)
sayaç; toplayıcı	counter; accumulator	Turları sayan; değer biriktiren değişken
sıra; seçim; tekrar	sequence; selection; iteration	Üç temel kontrol yapısı
izleme tablosu	trace table	Algoritmanın elle, adım adım yürütüldüğü tablo
test durumu; sınır değer	test case; boundary value	Girdi + beklenen çıktı; aralığın kenarındaki girdi
bir-fark hatası	off-by-one error	Döngünün bir tur eksik ya da fazla çalışması
koşullu atlama	conditional jump	Bayraklara göre PC'yi değiştiren komut (ör. jle)

Şekil 2.19
Hafta 2 kavram haritası



Gelecek hafta (H3): algoritmadan C'ye. Kutuların bellekteki gerçek hâli: bir `int` kaç bayt kaplar, `-7` nasıl saklanır, `0.1 + 0.2` neden tam `0.3` değildir, faktöriyel neden 13'te taşar?

12. Kaynaklar

Böhm, C., & Jacopini, G. (1966). Flow diagrams, Turing machines and languages with only two formation rules. *Communications of the ACM*, *9*(5), 366–371. https://doi.org/10.1145/355592.365646

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Deitel, P., & Deitel, H. (2022). *C how to program: With case studies introducing applications and systems programming* (9th ed., global ed.). Pearson.

Dijkstra, E. W. (1968). Go to statement considered harmful. *Communications of the ACM*, *11*(3), 147–148. https://doi.org/10.1145/362929.362947

ISO. (1985). *Information processing — Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts* (ISO Standard No. 5807:1985). International Organization for Standardization.

Knuth, D. E. (1997). *The art of computer programming: Vol. 1. Fundamental algorithms* (3rd ed.). Addison-Wesley.

Lister, R., Adams, E. S., Fitzgerald, S., Fone, W., Hamer, J., Lindholm, M., McCartney, R., Moström, J. E., Sanders, K., Seppälä, O., Simon, B., & Thomas, L. (2004). A multi-national study of reading and tracing skills in novice programmers. *ACM SIGCSE Bulletin*, *36*(4), 119–150. https://doi.org/10.1145/1041624.1041673

Pólya, G. (1945). *How to solve it: A new aspect of mathematical method*. Princeton University Press.

Recep Tayyip Erdoğan Üniversitesi. (2025). *Recep Tayyip Erdoğan Üniversitesi ön lisans ve lisans eğitim-öğretim ve sınav yönetmeliği* (Resmî Gazete, 14 Aralık 2025, Sayı: 33107; son değişiklik: Resmî Gazete, 27 Şubat 2026, Sayı: 33181).

Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, *49*(3), 33–35. https://doi.org/10.1145/1118178.1118215

Wirth, N. (1971). Program development by stepwise refinement. *Communications of the ACM*, *14*(4), 221–227. https://doi.org/10.1145/362575.362577